Phase 5: 基础模块实现指导

本文档提供实现每个基础模块时需要了解的 PyTorch 语法、模式和注意事项。 **不包含实际实现代码**——你需要自己将这些知识组合起来完成实现。

---

1. nn.Module 基础语法

1.1 定义自定义 Module 的模板

```
import torch
import torch.nn as nn

class MyModule(nn.Module):
    def __init__(self, ...):
        super().__init__()          # 必须调用父类构造函数
        # 在这里定义参数和子模块
        self.weight = nn.Parameter(torch.empty(...))
        # 初始化
        nn.init.trunc_normal_(self.weight, mean=0, std=..., a=-3*std,
b=3*std)

    def forward(self, x):
        # 计算逻辑
        return result
```

1.2 nn.Parameter vs register_buffer

- `nn.Parameter(tensor)`：可训练参数，会出现在 `model.parameters()` 中，被优化器更新
- `self.register_buffer("name", tensor, persistent=False)`：不被优化器更新的张量
  - `persistent=True`（默认）：会被 `state_dict()` 保存
  - `persistent=False`：不被保存，每次重新计算

1.3 trunc_normal_ 初始化

```
# 截断正态分布初始化
# mean: 均值
# std: 标准差
# a, b: 截断范围
torch.nn.init.trunc_normal_(tensor, mean=0.0, std=1.0, a=-2.0, b=2.0)

# 例: Linear 权重初始化
# σ = sqrt(2 / (d_in + d_out))
# 截断到 [-3σ, 3σ]
import math
std = math.sqrt(2.0 / (d_in + d_out))
nn.init.trunc_normal_(self.weight, mean=0.0, std=std, a=-3*std, b=3*std)
```

```
# Embedding 初始化
# σ = 1, 截断到 [-3, 3]
nn.init.trunc_normal_(self.weight, mean=0.0, std=1.0, a=-3.0, b=3.0)

# RMSNorm 初始化为全 1
nn.init.ones_(self.weight)
# 或直接 self.weight = nn.Parameter(torch.ones(d_model))
```

---

2. Softmax 实现要点

2.1 需要的 PyTorch 操作

```
# 沿指定维度取最大值（keepdim=True 保持维度不塌缩）
x_max = x.max(dim=dim, keepdim=True).values

# 或等价的
x_max = torch.amax(x, dim=dim, keepdim=True)

# 指数
torch.exp(x)

# 求和
torch.sum(x, dim=dim, keepdim=True)
```

2.2 数值稳定性

数学上 `softmax(x) = softmax(x - c)` 对任意常数 c 成立。 取 c = max(x) 可以防止 exp 溢出（因为 exp 的最大输入变为 0）。

2.3 广播语义

当 `dim=-1` 且 x 形状为 `(3, 4, 5)` 时：

- `x.max(dim=-1, keepdim=True)` → 形状 `(3, 4, 1)`
- `x - x_max` 利用广播自动在最后一维展开
- 这对任意形状的输入都有效

---

3. Linear 实现要点

3.1 核心计算

`y = x @ W^T` 其中 W 的形状是 `(d_out, d_in)`

3.2 PyTorch 矩阵乘法语义

```
# @ 运算符对高维张量有 batched matmul 语义
# x: (B, T, d_in)  W: (d_out, d_in)  W.T: (d_in, d_out)
# x @ W.T → (B, T, d_out)
# PyTorch 自动广播前面的维度

# 或用 einsum（更清晰）
# torch.einsum('...i, oi -> ...o', x, W)

# .T 是 .transpose(-2, -1) 的简写（只对 2D 张量）
# 对高维张量用 .mT 或 .transpose(-2, -1)
```

3.3 device 和 dtype 参数

```
def __init__(self, in_features, out_features, device=None, dtype=None):
    super().__init__()
    # 创建空张量时指定 device/dtype
    self.weight = nn.Parameter(
        torch.empty(out_features, in_features, device=device, dtype=dtype)
    )
    # 然后初始化
    nn.init.trunc_normal_(self.weight, ...)
```

这允许直接在 GPU 上创建参数，避免不必要的 CPU→GPU 拷贝。

---

4. Embedding 实现要点

4.1 核心计算

Embedding 就是一个查表操作：给定整数索引，返回表中对应行。

4.2 PyTorch 高级索引

```
# weight 形状: (vocab_size, d_model)
# token_ids: 任意形状的整数张量
# weight[token_ids] 返回形状为 (*token_ids.shape, d_model) 的张量

# 例:
E = torch.randn(100, 16)  # 100 个 token, 16 维
ids = torch.tensor([[1, 2, 3], [4, 5, 6]])  # (2, 3)
E[ids]  # → (2, 3, 16)
```

这是因为 PyTorch 的整数索引会沿第 0 维选取对应行，保留索引张量的形状。

---

5. RMSNorm 实现要点

5.1 dtype upcast 模式

```
def forward(self, x):
    # 记住原始 dtype
    orig_dtype = x.dtype

    # upcast 到 float32 防止溢出
    x_float = x.to(torch.float32)

    # 计算 RMS（沿最后一维）
    # mean(x², dim=-1, keepdim=True) 然后加 eps 再开根号

    # 归一化
    # x_normed = x_float / rms

    # 应用可学习缩放
    # output = x_normed * self.weight

    # cast 回原 dtype
    return output.to(orig_dtype)
```

5.2 相关 PyTorch 操作

```
# 平方
x ** 2
# 或
torch.square(x)

# 沿最后一维求均值
torch.mean(x, dim=-1, keepdim=True)

# 开根号
torch.sqrt(x)

# dtype 转换
x.to(torch.float32)
x.to(orig_dtype)

# 也可以用 .float() 和 .to(orig_dtype)
```

---

6. SiLU 实现要点

6.1 公式

```
SiLU(x) = x * σ(x) = x * sigmoid(x)
```

6.2 直接使用 torch.sigmoid

```
torch.sigmoid(x)  # 允许使用，不在 nn.functional 中
```

作业说明明确允许使用 `torch.sigmoid` 来保证数值稳定性。

---

7. 常用 PyTorch 技巧总结

7.1 einsum 记号参考

```
# 矩阵乘法: C = A @ B
torch.einsum('ij, jk -> ik', A, B)

# batch 矩阵乘法: (B, M, K) × (B, K, N) → (B, M, N)
torch.einsum('bmk, bkn -> bmn', A, B)

# 带任意 batch 维度的矩阵乘法:
torch.einsum('...ij, jk -> ...ik', x, W)

# 向量点积:
torch.einsum('i, i ->', a, b)

# batch 向量内积:
torch.einsum('bi, bi -> b', a, b)
```

7.2 形状操作

```
# view vs reshape:
# view 要求内存连续，reshape 不要求（可能产生拷贝）
x.view(B, H, T, dk)
x.reshape(B, H, T, dk)

# transpose: 交换两个维度
x.transpose(1, 2)   # 交换 dim 1 和 dim 2

# contiguous: 确保内存连续（transpose 后通常需要）
x.transpose(1, 2).contiguous()

# einops.rearrange（推荐，更可读）:
from einops import rearrange
rearrange(x, 'b t (h dk) -> b h t dk', h=num_heads)
rearrange(x, 'b h t dk -> b t (h dk)')
```

7.3 张量创建

```
# 空张量（未初始化）
torch.empty(shape, device=device, dtype=dtype)

# 全零 / 全一
torch.zeros(shape), torch.ones(shape)

# 随机正态
torch.randn(shape)

# 指定范围的整数
torch.arange(start, end)  # [start, end)
torch.arange(end)         # [0, end)

# bool mask
torch.ones(n, n, dtype=torch.bool)
torch.tril(torch.ones(n, n, dtype=torch.bool))
```

7.4 原地操作

```
# 带下划线的版本是原地操作
x.mul_(scalar)      # x *= scalar
x.add_(y)           # x += y
x.zero_()           # x = 0
x.fill_(value)      # x = value

# 赋值给 .data 不会被 autograd 追踪
param.data = new_tensor
param.data.mul_(0.99)  # 原地修改参数值
```